République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique Université de Sousse

Institut Supérieur des Sciences Appliquées et de Technologie de Sousse

DEPARTEMENT INFORMATIQUE

RAPPORT DE MINI PROJET WEB

ING A2-G2-2

Quiz interactif en ligne

Élaboré par :

Étudiant 1 : Chaima Saadana

Étudiant 2 : Fedi Zarai

A.U. : 2026/2027

Plan

1. Introduction
2. Cahier des charges / Objectifs
3. Conception
4. Implémentation (code et explications)
5. Résultat final
6. Difficultés rencontrées et solutions
7. Conclusion
8. Annexes

1. Introduction

Ce rapport présente le mini-projet réalisé dans le cadre du TP de développement web (INGA2, département informatique de l'ISSATSo, année universitaire 2026/2027). Parmi les sujets proposés, nous avons choisi le sujet 7 : une page de quiz interactif.

L'objectif est de concevoir une interface web ludique qui pose une série de questions à l'utilisateur, corrige chaque réponse immédiatement par un retour visuel, puis affiche un score final. Le projet utilise HTML5 pour la structure, CSS3 pour la mise en forme et les animations, et JavaScript pour la logique du quiz. Ce rapport décrit le cahier des charges, la conception, l'implémentation, le résultat obtenu et les difficultés rencontrées.

2. Cahier des charges / Objectifs

Le sujet demande de créer une interface web ludique permettant de répondre à un quiz et d'afficher les résultats en temps réel avec un retour visuel. Les contraintes techniques imposées sont les suivantes :

- HTML5, CSS3 et JavaScript ;
- design responsive avec Flexbox et Grid ;
- accessibilité et animations CSS ;
- validation W3C du code.

Les livrables attendus sont une page d'accueil et une interface de quiz, un retour visuel et un score final, des fichiers HTML, CSS et JS séparés, ainsi qu'une version responsive testée.

3. Conception

Structure du dossier

```
/projet-quiz
├── index.html
├── style.css
└── script.js
```

Les trois fichiers sont séparés conformément au sujet : index.html contient la structure, style.css la mise en forme et script.js la logique du quiz.

Structure HTML

La page est organisée logiquement : un lien d'évitement pour l'accessibilité, un élément main qui contient trois sections (accueil, quiz, résultat) dont une seule est visible à la fois, puis un footer et l'appel au script. Le JavaScript affiche ou masque chaque section grâce à l'attribut hidden.

Charte graphique

Nous avons choisi une ambiance ludique et lisible : un violet dominant, des cartes blanches aux coins arrondis, et des couleurs vives et sans ambiguïté pour le retour visuel. Les polices sont Baloo 2 pour les titres et Nunito pour le texte, toutes deux issues de Google Fonts.

Couleur	Code	Usage
Violet principal	#5B3DF5	Titres, boutons, barre de progression
Fond de page	#F1EFFC	Arrière-plan général
Vert	#12795A	Bonne réponse
Rouge	#B3203D	Mauvaise réponse
Jaune	#FFC83D	Contour de focus au clavier

Outils utilisés

- Visual Studio Code avec l'extension Live Server ;
- navigateur Chrome et ses outils de développement (mode appareil mobile) ;
- validateurs W3C pour le HTML (validator.w3.org) et le CSS (jigsaw.w3.org/css-validator) ;
- Google Fonts pour les polices.

4. Implémentation (code et explications)

Exemple de code HTML

Voici la section du quiz. Elle contient le compteur et le score, la barre de progression (avec les attributs ARIA), la question, la liste des réponses remplie par JavaScript, le message de correction et le bouton de passage à la question suivante.

```
<section id="quiz" class="ecran" hidden>
    <div class="haut">
        <p id="compteur">Question 1 sur 8</p>
        <p id="score-live">Score : 0</p>
    </div>
    <div class="barre" role="progressbar" aria-label="Progression du quiz"
         aria-valuemin="0" aria-valuemax="8" aria-valuenow="0">
        <div id="remplissage"></div>
    </div>
    <h2 id="question" tabindex="-1">Chargement de la question</h2>
    <ul id="reponses" class="reponses"></ul>
    <p id="retour" class="retour" role="status"></p>
    <button id="suivante" class="bouton" type="button" hidden>
        Question suivante</button>
</section>
```

L'attribut role="status" permet aux lecteurs d'écran d'annoncer la correction, et tabindex="-1" permet de placer le focus sur la question à chaque changement.

Exemple de code CSS

Les réponses sont disposées avec CSS Grid sur deux colonnes. Le retour visuel combine la couleur, une icône (✓ ou ✗) et une animation, pour ne pas dépendre de la couleur seule. Sur mobile, une media query passe la grille sur une seule colonne.

```
.reponses {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 12px;
    margin: 0; padding: 0; list-style: none;
}

.reponse.juste { border-color: var(--juste); background: var(--juste-fond);
                 color: var(--juste); animation: rebond .35s; }
.reponse.juste::after { content: "✓"; font-size: 1.4rem; }
.reponse.faux  { border-color: var(--faux); background: var(--faux-fond);
                 color: var(--faux); animation: secousse .4s; }
.reponse.faux::after  { content: "✗"; font-size: 1.4rem; }

@keyframes secousse { 25% { transform: translateX(-6px); }
                      75% { transform: translateX(6px); } }

@media (max-width: 600px) {
    .reponses { grid-template-columns: 1fr; }
}
```

Exemple de code JavaScript

La fonction repondre() est appelée quand l'utilisateur clique sur une réponse. Elle empêche une seconde réponse, met à jour le score, colore la bonne et la mauvaise réponse, puis affiche l'explication.

```
function repondre(bouton, courante) {
    if (repondu) return;
    repondu = true;

    const juste = bouton.dataset.juste === "true";
    if (juste) score++;

    document.querySelectorAll(".reponse").forEach((b) => {
        b.classList.add("verrouille");
        if (b.dataset.juste === "true") b.classList.add("juste");
    });
    if (!juste) bouton.classList.add("faux");

    $("retour").textContent = juste
        ? `Bonne réponse ! ${courante.explication}`
        : `Raté. La bonne réponse était : ${courante.options[courante.bonne]}.`;
    $("score-live").textContent = `Score : ${score}`;
    $("suivante").hidden = false;
}
```

Les questions sont stockées dans un tableau JavaScript (texte, quatre options, indice de la bonne réponse et explication). L'ordre des questions et des réponses est mélangé à chaque partie, ce qui rend le quiz rejouable.

5. Résultat final

La page d'accueil présente le quiz en une carte centrée : titre, description, informations clés et bouton de démarrage.



Figure 1 : page d'accueil

Pendant le quiz, le compteur, le score et la barre de progression se mettent à jour à chaque question. Une bonne réponse s'affiche en vert avec une coche et un message d'encouragement.



Figure 2 : bonne réponse

En cas d'erreur, la réponse choisie passe en rouge avec une croix, la bonne réponse est mise en évidence en vert, et le message indique la solution avec une courte explication.



Figure 3 : mauvaise réponse et correction

À la fin, un cercle de progression en CSS affiche le score, accompagné d'un message adapté au résultat et d'un bouton pour rejouer.

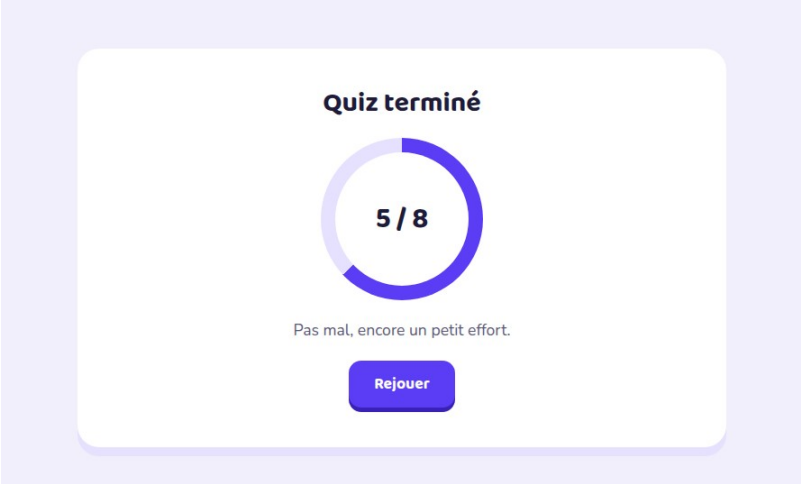


Figure 4 : score final

La mise en page s'adapte à la taille de l'écran : les réponses passent sur une seule colonne sur mobile, et la carte reste centrée et lisible sur tablette. Le rendu reste cohérent entre les trois tailles d'écran (même charte graphique, mêmes composants).

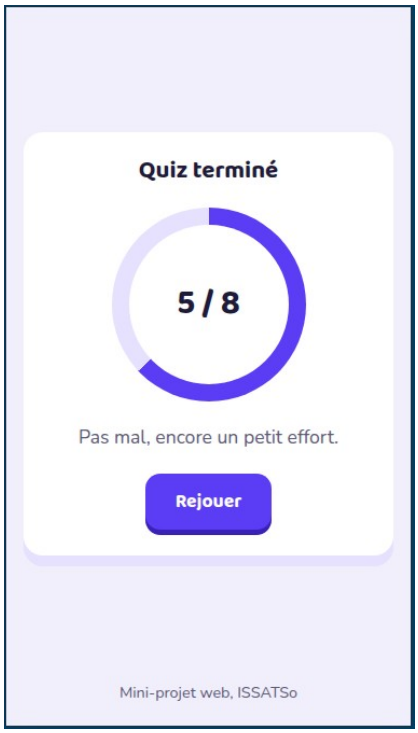


Figure 5 : version mobile (375 px)

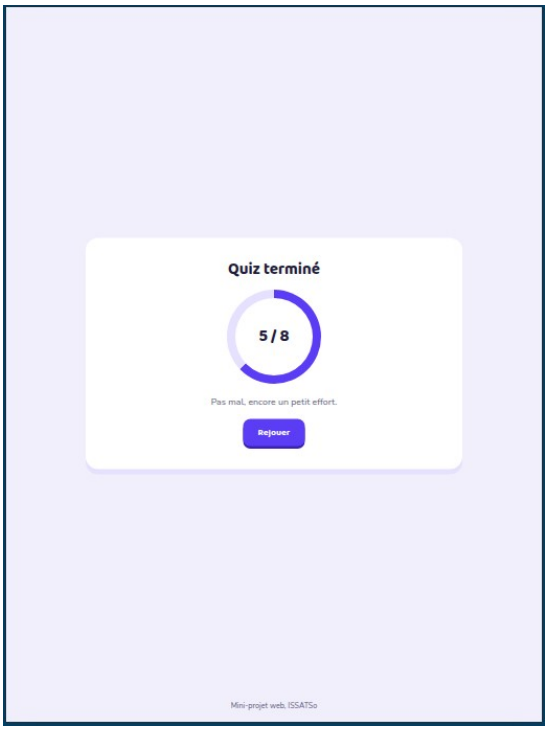


Figure 6 : version tablette (iPad Pro)

Enfin, les fichiers ont été vérifiés avec les validateurs W3C. Le CSS est valide, sans aucune erreur :



Figure 7 : validation W3C du CSS

Le validateur HTML ne signale aucune erreur, seulement un avertissement sur un titre vide, corrigé ensuite (voir la section suivante) :

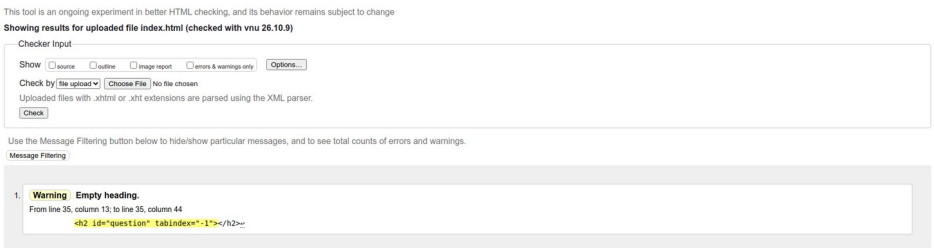


Figure 8 : validation W3C du HTML (avertissement avant correction)

6. Difficultés rencontrées et solutions

- Avertissement W3C « Empty heading » : la balise <h2 id="question"> était vide dans le HTML car elle est remplie par JavaScript. Problème résolu en ajoutant un texte par défaut dans la balise, aussitôt remplacé par le script.
- Rendu des réponses sur mobile : deux colonnes étaient trop étroites sur petit écran. Corrigé avec une media query (max-width: 600px) qui passe la grille sur une colonne.

- Retour visuel accessible : une couleur seule ne suffit pas à distinguer les bonnes et mauvaises réponses. Résolu en ajoutant des icônes (✓ et ✗) et un message texte annoncé par role="status".
- Réponses multiples : un second clic pouvait modifier le score. Résolu avec une variable (repondu) qui verrouille la question après la première réponse.

7. Conclusion

Ce projet nous a permis de consolider nos connaissances en HTML, CSS et JavaScript : structuration d'une page en plusieurs écrans, mise en page avec Flexbox et Grid, conception responsive, animations CSS et manipulation du DOM. Nous avons aussi appris à tenir compte de l'accessibilité (navigation au clavier, focus visible, annonces pour lecteurs d'écran) et à valider le code avec les outils du W3C.

Il constitue une base solide pour aborder le développement web dynamique : le quiz pourrait par exemple être amélioré avec une base de questions chargée depuis un serveur.

8. Annexes

- Code complet du projet : fichiers index.html, style.css et script.js (dossier projet-quiz).
- Validateur HTML du W3C : https://validator.w3.org
- Validateur CSS du W3C : https://jigsaw.w3.org/css-validator
- Google Fonts (Baloo 2, Nunito) : https://fonts.google.com
- Documentation web MDN : https://developer.mozilla.org